

第4章 串

提纲

CONTENTS



4.1 串的基本概念

4.2 串的存储结构

4.3 串的模式匹配

4.1 串的基本概念

串（或字符串）是由零个或多个**字符**组成的有限序列。

串 $\subset$ 线性表

- 串中所含字符的个数称为该**串的长度**（或串长）。
- 含零个字符的串称为**空串**，用 Φ 表示。

4.1 串的基本概念

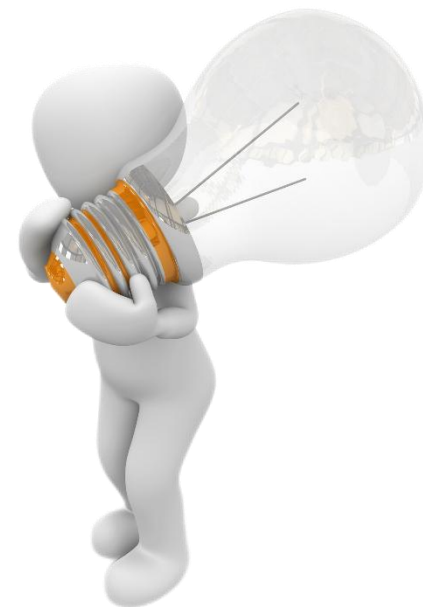
串的逻辑表示:

“ $a_1a_2\cdots a_n$ ”



双引号不是串的内容, 起标识作用

串的逻辑表示, a_i ($1 \leq i \leq n$) 代表一个字符。



4.1 串的基本概念

串相等：两个串的长度相等并且各个对应位置上的字符都相同时。

如：

“abcd” **$\neq$** ***“abc”***

“abcd” **$\neq$** ***“abcde”***

所有空串是相等的。



4.1 串的基本概念

子串:

一个串中任意个连续字符组成的子序列（含空串）称为该串的子串。

例如, “abcde”的子串有: “”、“a”、“ab”、“abc”、“abcd”和“abcde”等

真子串是指不包含自身的所有子串。



4.1 串的基本概念

串抽象数据类型 = 逻辑结构 + 基本运算 (运算描述)

串的基本运算如下:

- ① **StrAssign(&s, cstr):** 将字符串常量cstr赋给串s, 即生成其值等于cstr的串s。
- ② **StrCopy(&s, t):** 串复制。将串t赋给串s。
- ③ **StrEqual(s, t):** 判串相等。若两个串s与t相等则返回真; 否则返回假。
- ④ **StrLength(s):** 求串长。返回串s中字符个数。
- ⑤ **Concat(s, t):** 串连接:返回由两个串s和t连接在一起形成的新串。
- ⑥ **SubStr(s, i, j):** 求子串。返回串s中从第i ($1 \leq i \leq n$) 个字符开始的、由连续j个字符组成的子串。

4.1 串的基本概念

- ⑦ **InsStr(s1, i, s2):** 插入。将串s2插入到串s1的第i ($1 \leq i \leq n+1$) 个字符中, 即将s2的第一个字符作为s1的第i个字符, 并返回产生的新串。
- ⑧ **DelStr(s, i, j):** 删除。从串s中删去从第i ($1 \leq i \leq n$) 个字符开始的长度为j的子串, 并返回产生的新串。
- ⑨ **RepStr(s, i, j, t):** 替换。在串s中, 将第i ($1 \leq i \leq n$) 个字符开始的j个字符构成的子串用串t替换, 并返回产生的新串。
- ⑩ **DispStr(s):** 串输出。输出串s的所有元素值。

串A= “abcdefg” , B= “xx” ,

给出 (1) DelStr (A,3,2)

“abefg”

(2)RepStr (A, 3, 2, B)

“abxxefg”

第4章 串

提纲

CONTENTS

4.1 串的基本概念

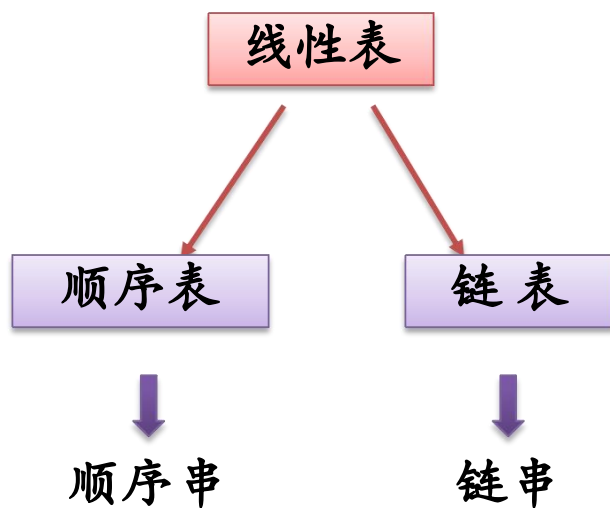
▶ 4.2 串的存储结构

4.3 串的模式匹配

4.1 串的基本概念

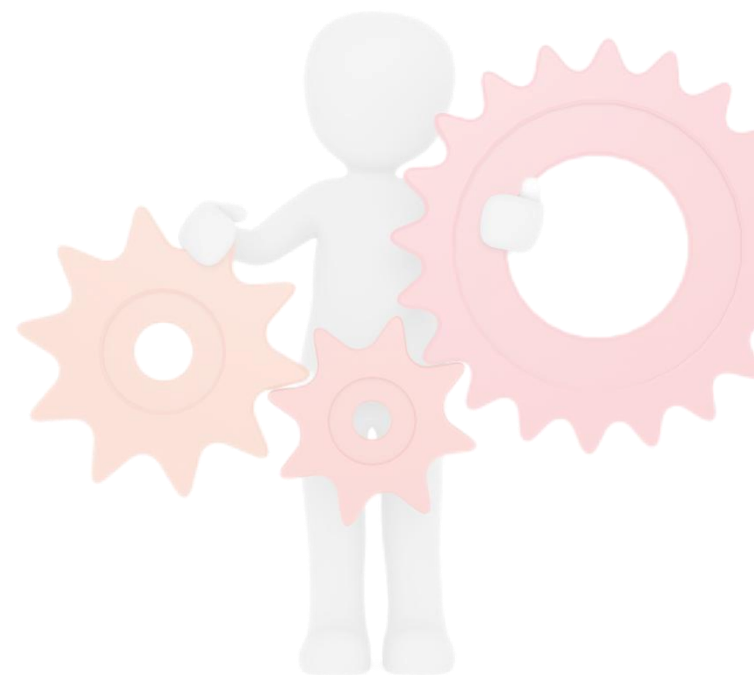
4.2.1 串的顺序存储及其基本操作实现

串中元素逻辑关系与线性表的相同，串可以采用与线性表相同的存储结构。



逻辑结构

存储结构



4.1 串的基本概念

4.2.1 串的顺序存储及其基本操作实现

串的顺序存储（顺序串）有两种方法：

- 每个单元(如4个字节)只存一个字符，称为**非紧缩格式**（其存储密度小）。
- 每个单元存放多个字符，称为**紧缩格式**（其存储密度大）。

1001	A			
1002	B			
1003	C			
1004	D			
1005	E			
1006	F			
1007	G			
1008	H			
1009	I			
100a	J			

非紧缩格式示例

1001	A	B	C	D
1002	E	F	G	H
1003	I	J		

紧缩格式示例

一个单元



4.1 串的基本概念

4.2.1 串的顺序存储及其基本操作实现

对于非紧凑格式（一个**char**存放一个字符）的顺序串，其类型声明如下：

```
typedef struct
```

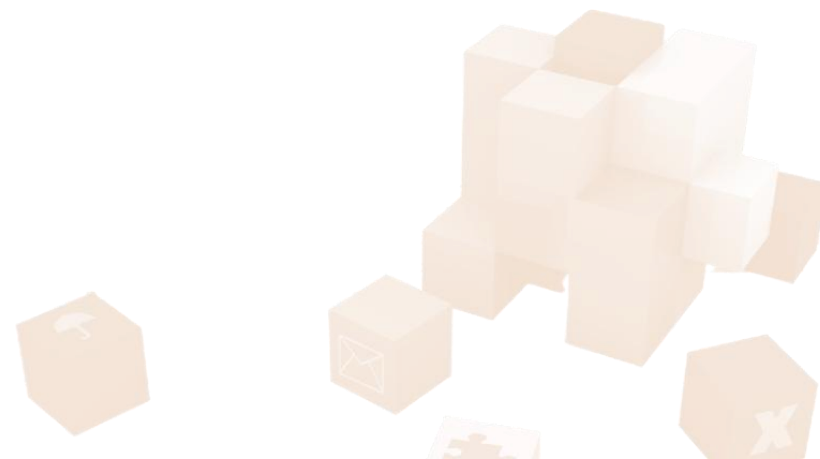
```
{  char data[MaxSize];
```

```
    int length;
```

```
}  SqString;
```

用来存储字符串

用来存储字符串长度



4.1 串的基本概念

4.2.1 串的顺序存储及其基本操作实现

顺序串中实现串的基本运算与顺序表的基本运算类似。详细算法实现参见第2章顺序表部分。



4.1 串的基本概念

4.2.1 串的顺序存储及其基本操作实现

1) 生成串 StrAssign(&s, cstr)

将一个C/C++字符串常量cstr（以'\0'字符标识结尾）赋给顺序串s，即生成一个其值等于cstr的串s。

```
void StrAssign(SqString &s,char cstr[]) //s为引用型参数
{ int i;
  for (i=0;cstr[i]!='\0';i++)
    s.data[i]=cstr[i];
  s.length=i; //设置串s的长度
}
```

2) 销毁串 DestroyStr(&s)

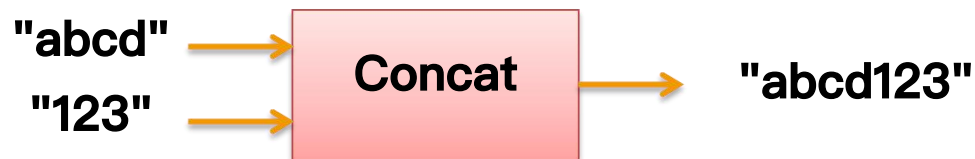
```
void DestroyStr(SqString &s)
{ free(s) }
```

4.1 串的基本概念

4.2.1 串的顺序存储及其基本操作实现

6) 串连接Concat(s, t)

返回由两个顺序串s和t连接在一起形成的结果串。



```
SqString Concat(SqString s, SqString t)
{
    SqString str;           // 定义结果串
    int i;
    str.length = s.length + t.length; // 计算串str的长度
    for (i = 0; i < s.length; i++) // 将s.data[0..s.length-1]复制到str
        str.data[i] = s.data[i];
    for (i = 0; i < t.length; i++) // 将t.data[0..t.length-1]复制到str
        str.data[s.length + i] = t.data[i];
    return str;
}
```

4.1 串的基本概念

4.2.1 串的顺序存储及其基本操作实现

【例4.1】 设计顺序串上实现串比较运算Strcmp(s, t)的算法。例如：

"ab" < "abcd" "abcd" < "abd"

解

(1) 比较s和t两个串**共同长度范围内的**对应字符：

① 若s的字符>t的字符，返回1；

② 若s的字符<t的字符，返回-1；

③ 若s的字符=t的字符，按上述规则继续比较。

(2) 当 (1) 中对应字符均相同时，比较s和t的长度：

① 两者相等时，返回0；

② s的长度>t的长度，返回1；

③ s的长度<t的长度，返回-1。

"ba">"abc"

"aba"<"bc"

"abc"="abc"

"abc">"ab"

"ab"<"abd"

4.1 串的基本概念

4.2.1 串的顺序存储及其基本操作实现

```
int Strcmp(SqString s, SqString t)
{ int i, comlen;
  if (s.length < t.length)
    comlen = s.length;           //求s和t的共同长度
  else
    comlen = t.length;
  for (i = 0; i < comlen; i++)    //在共同长度内逐个字符比较
    if (s.data[i] > t.data[i])
      return 1;
    else if (s.data[i] < t.data[i])
      return -1;
  if (s.length == t.length)      //前面已经判断过字符的大小了，这里长度相等就表示s==t
    return 0;
  else if (s.length > t.length)  //s>t
    return 1;
  else return -1;                //s<t
}
```


4.2 串的存储结构

4.2.2 串的链式存储及其基本操作实现

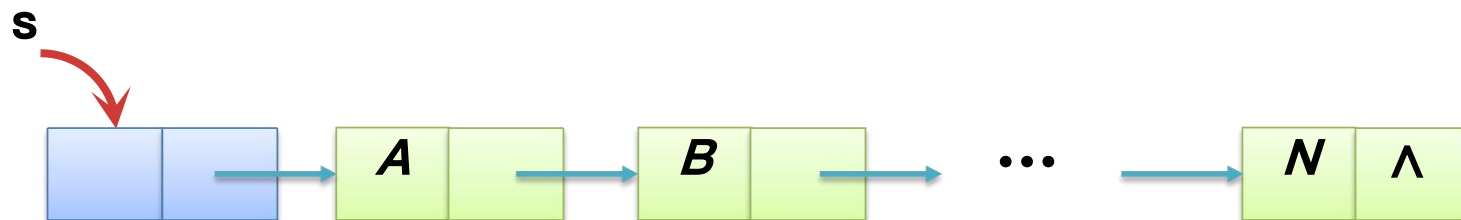
- 链串的组织形式与一般的链表类似。
- 链串中的一个结点可以存储多个字符。
通常将链串中每个结点所存储的字符个数称为结点大小。

4.2 串的存储结构

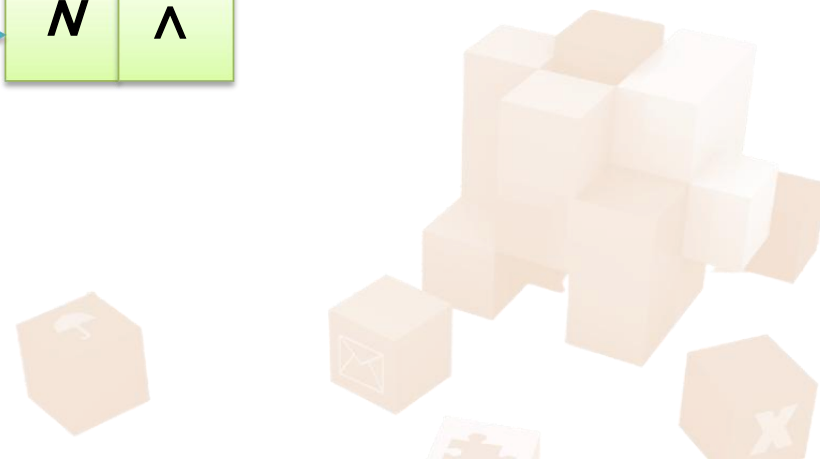
4.2.2 串的链式存储及其基本操作实现



结点大小为4的链串



结点大小为1的链串

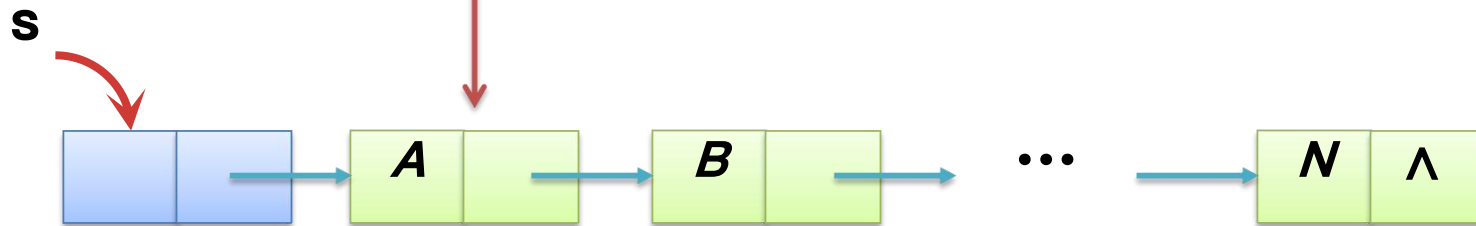


4.2 串的存储结构

4.2.2 串的链式存储及其基本操作实现

链串结点大小1时，链串的结点类型声明如下：

```
typedef struct snode
{
    char data;
    struct snode *next;
} LinkStrNode;
```



4.2 串的存储结构

4.2.2 串的链式存储及其基本操作实现



链串中实现串的基本运算

与单链表的基本运算类似。

4.2 串的存储结构

4.2.2 串的链式存储及其基本操作实现

【例4.3】 在链串中，设计一个算法把最先出现的子串“*ab*”改为“*xyz*”。

解

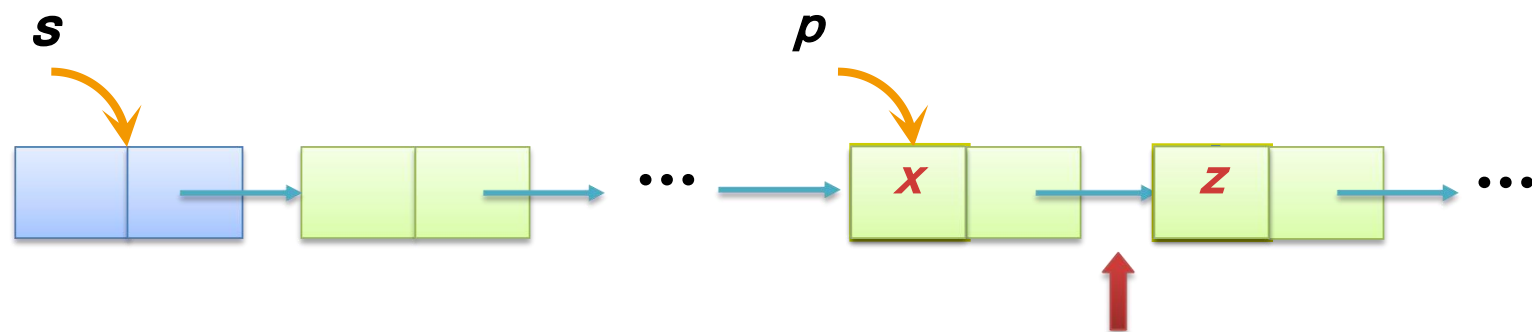
① 查找: $p \rightarrow \text{data} = 'a' \ \&\& \ p \rightarrow \text{next} \rightarrow \text{data} = 'b'$



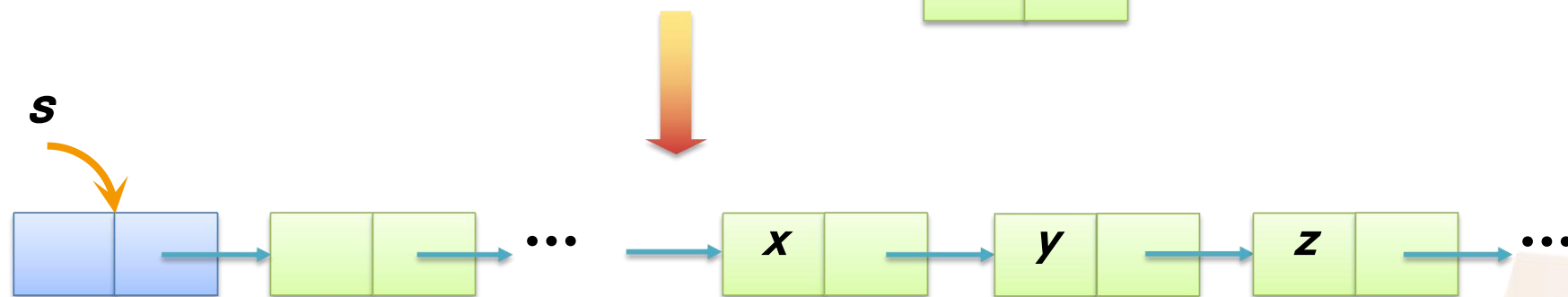
4.2 串的存储结构

4.2.2 串的链式存储及其基本操作实现

② 替换



③ 插入



4.2 串的存储结构

4.2.2 串的链式存储及其基本操作实现

```
void Repl(LinkStrNode *&s)
{
    LinkStrNode *p=s->next, *q;
    int find=0;
    while (p->next!=NULL && find==0)           //查找ab子串
    {
        if (p->data=='a' && p->next->data=='b')
        { p->data='x'; p->next->data='z';        //xz替换ab
          q=(LinkStrNode *)malloc(sizeof(LinkStrNode));
          q->data='y'; q->next=p->next; p->next=q; //将y插入
          find=1;
        }
        else p=p->next;
    }
}
```

算法的时间复杂度为 $O(n)$ 。

替换为xyz

4.2 串的存储结构

4.2.2 串的链式存储及其基本操作实现

链串只能采用单链表吗？

- **不一定**。需要根据需要情况而定。
- 如果需要从某个结点出发前后查找，可以采用双链表。
- 如果需要快速查找尾结点，可以采用循环双链表。

第4章 串

提纲

CONTENTS

4.1 串的基本概念

4.2 串的存储结构

▶ 4.3 串的模式匹配

4.3 串的模式匹配

4.3.1 Brute-Force算法



- **成功**是指在目标串 s 中找到一个模式串 t —— t 是 s 的子串，返回 t 在 s 中的位置。
- **不成功**则指目标串 s 中不存在模式串 t —— t 不是 s 的子串，返回 -1。

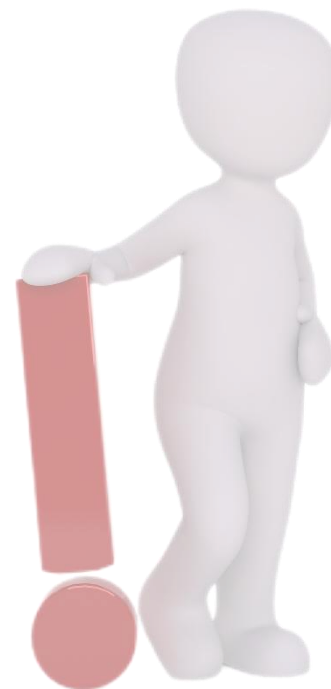
4.3 串的模式匹配

4.3.1 Brute-Force算法

Brute-Force简称为**BF算法**，亦称简单匹配算法。采用穷举的思路。

BF是指暴力的意思！

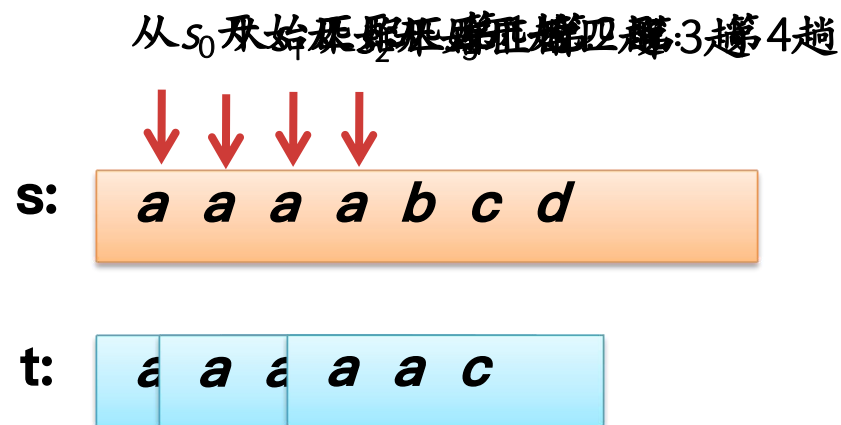
例如， $s = \text{"aaaabcd"}$ ， $t = \text{"aac"}$ 。



4.3 串的模式匹配

4.3.1 Brute-Force算法

BF算法思路： 从s的每一个字符开始依次与t的字符进行匹配。



过程

- i, j 分别扫描字符串 s 和 t
- i 从 0 到 $s.length - t.length$ 循环。
- 对于每个 $s[i]$ ，从 $j=0$ 开始比较
- 如果 t 扫描完毕，则返回 i 。表示 t 在 s 中的位置
- 若 s 全部比较完毕都没有返回，则返回 -1

4.3 串的模式匹配

4.3.1 Brute-Force算法

```
int index(SqString s, SqString t)
{
    int i, j, k;
    for (i=0; i<=s.length-t.length; i++) //i从0到s.length-t.length循环
    {
        for (k=i, j=0; k<s.length && j<t.length &&
              s.data[k]==t.data[j]; k++, j++);
        //对于每个s[i], 从j=0开始比较。判断s与t中的字符是否一致
        if (j==t.length) // 表示在s中找到了子串t
            return(i); //返回t在s中的位置
    }
    return(-1);
}
```

4.3 串的模式匹配

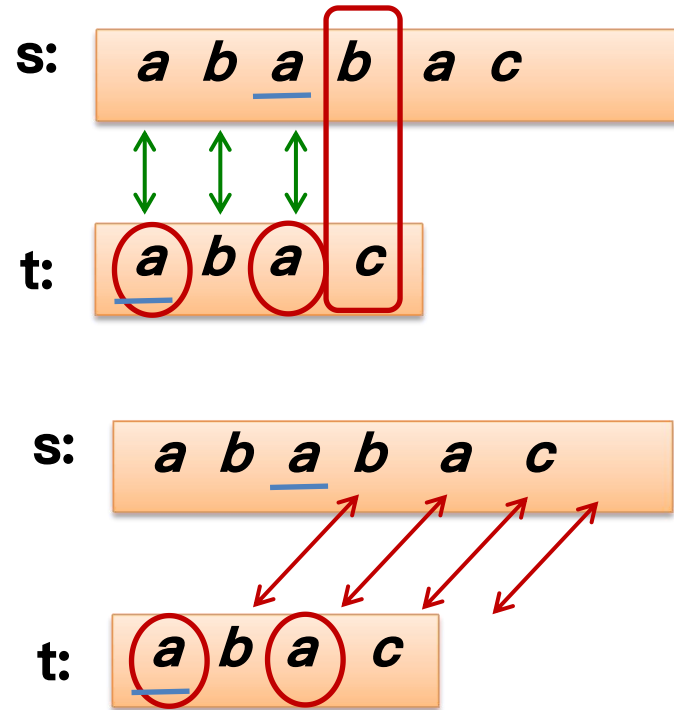
4.3.1 Brute-Force算法

BF算法分析

- 算法在字符比较不相等，需要回溯（即 $i=i-j+1$ ）：即退到s中的下一个字符开始进行继续匹配。
- 最好情况下的时间复杂度为 $O(m)$ 。
- 最坏情况下的时间复杂度为 $O(n \times m)$ 。
- 平均的时间复杂度为 $O(n \times m)$ 。

4.3 串的模式匹配

4.3.1 如何降低Brute-Force算法的时间复杂度？



4.3 串的模式匹配

4.3.2 KMP算法



KMP算法是D.E.Knuth、J.H.Morris和V.R.Pratt共同提出的，简称**KMP算法**。

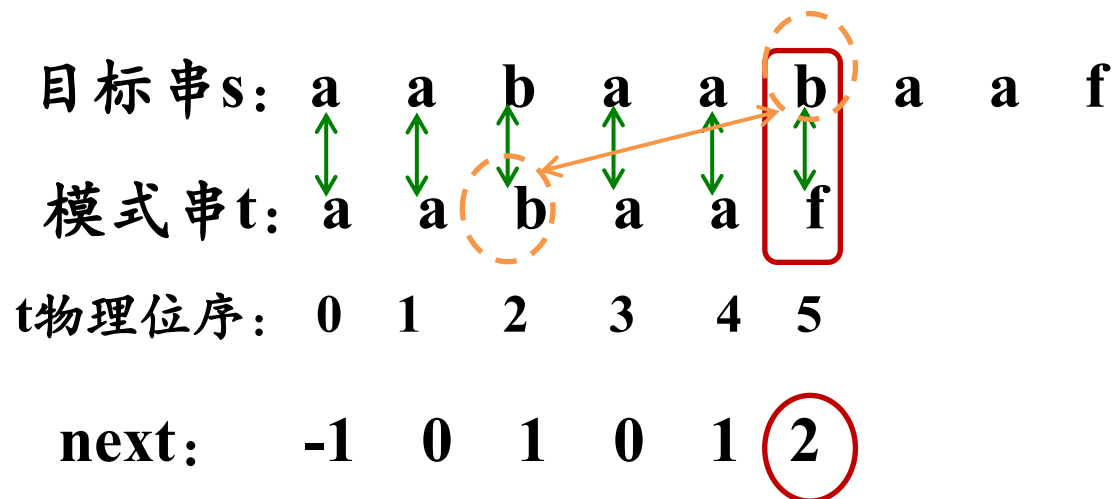
该算法较BF算法有较大改进，主要是**消除了主串指针的回溯**，从而使算法效率有了某种程度的提高。

采用空间换时间的方式，提取保存有利于匹配的信息。

4.3 串的模式匹配

如何得到next数组?

4.3.2 KMP算法



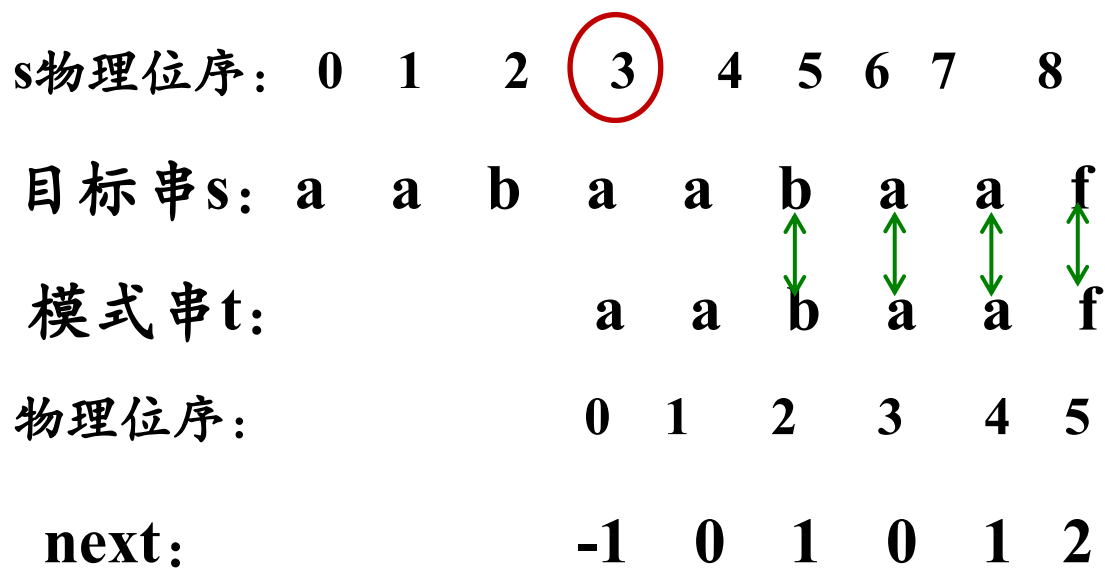
第一步：从头开始逐一匹配

第二步：遇到不匹配的字符时，找到对应next数组中相应物理位序中的值x。

第三步：将模式串t回溯到物理位序为x的位置，重新在刚刚目标串s不匹配的地方开始逐一匹配。

第四步：如果遇到不匹配的情况，重新执行第二第三步，直到遍历完目标串s。当next值为-1时，从目标串下一个开始匹配。

第五步：匹配上后返回目标串s中与模式串开始匹配的字符的物理位序。



4.3 串的模式匹配

4.3.2 KMP算法

1 $\text{next}[j]$ 是指 t_j 字符前**最多**有多少个字符与 t 开头的字符相同

模式串 t 中字符 t_j 存在某个 k ($0 < k < j$) 字符, 使得以下成立:

$$\underbrace{t_0 t_1 \cdots t_{k-1}}_{\text{开头的}k\text{个字符}} = \underbrace{t_{j-k} t_{j-k+1} \cdots t_{j-1}}_{t_j\text{前面的}k\text{个字符}}$$

最大相等前后缀的个数

求最大的 k

这个字符不能是 t_0

0 1 2 3 4

例如, $t = \text{"}a a a a c\text{"}$ 考虑 $t[4]='c'$

$t = \text{"}a a a a c\text{"}$



$\Rightarrow k=3$, 所以 $\text{next}[4] = k = 3$.

4.3 串的模式匹配

4.3.2 KMP算法

什么是前缀和后缀

字符串abcd

前缀：只包含首字母，不包含尾字符

a; ab; abc

后缀：只包含尾字符，不包含首字符

d; cd; bcd

4.3 串的模式匹配

4.3.2 KMP算法

2 next[0]的特殊含义

next[0]设置为-1

因为s的字符 s_i 与 t_0 不相同，没有任何有用的部分
匹配信息，直接从下一趟 (s_{i+1}/t_0) 开始匹配。



4.3 串的模式匹配

4.3.2 KMP算法

对于模式串 $t = "t_0 t_1 \cdots t_j \cdots t_{m-1}"$

归纳起来，定义 $next[j]$ 数组如下：

$$next[j] = \begin{cases} \text{MAX}\{ k \mid 0 < k < j, \text{ 且 } \overbrace{t_0 t_1 \cdots t_{k-1}}^{\text{开头的 } k \text{ 个字符}} = \overbrace{t_{j-k} t_{j-k+1} \cdots t_{j-1}}^{t_j \text{ 前面的 } k \text{ 个字符}} \} & \text{当此集合非空时} \\ -1 & \text{当 } j=0 \text{ 时} \\ 0 & \text{其他情况} \end{cases}$$

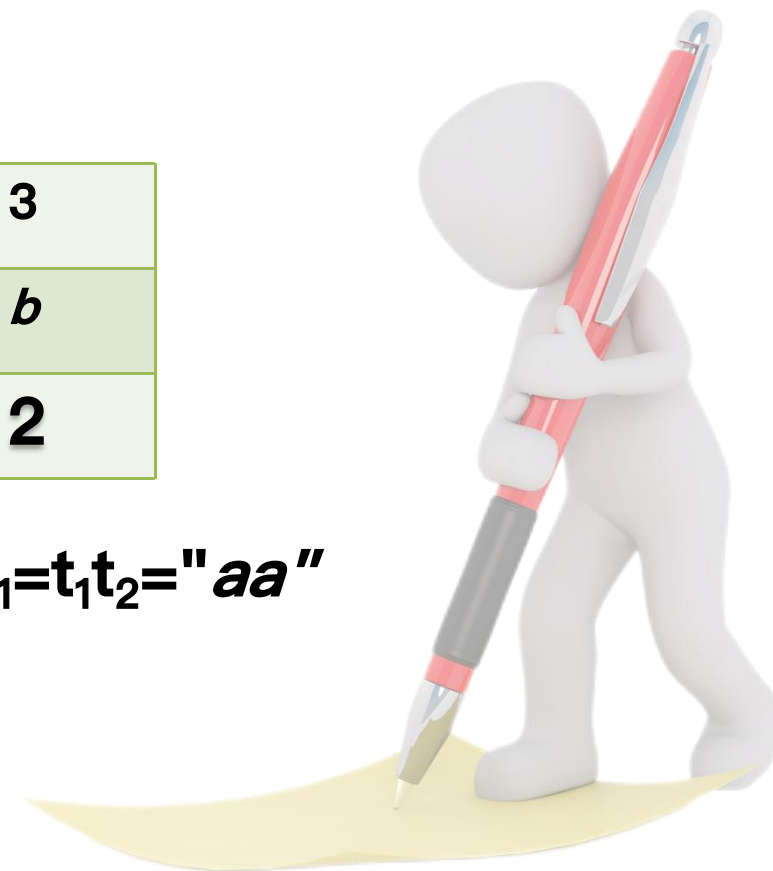
4.3 串的模式匹配

4.3.2 KMP算法

$t = \text{"aaab"}$ 对应的next数组如下:

j	0	1	2	3
t[j]	a	a	a	b
next[j]	-1	0	1	2

$t_0 = t_1 = \text{"a"}$ $t_0t_1 = t_1t_2 = \text{"aa"}$



题目：串‘ababaaaababaa’的next数组为（）

A. -1,0,1,2,3,4,5,6,7,8,8,8

B. -1,0,0,1,2,3,0,0,0,1,0,1

C. -1,0,0,1,2,3,1,1,2,3,4,5

D. -1,0,1,2,-1,0,1,2,1,1,2,3

答案： C

4.3 串的模式匹配

4.3.2 KMP算法

由模式串t求next值的算法:

```
void GetNext(SqString t, int next[])
{ int j, k;
  j=0; k=-1; next[0]=-1;
  while (j<t.length-1)
  { if (k==-1 || t.data[j]==t.data[k])
    { j++; k++;
      next[j]=k;
    }
    else
      k=next[k];
  }
}
```

t= "a a a b c"
next= -1 0 1 2 0

第1次循环:

j=0; k=-1

k=-1满足if条件

j+1=1; k+1=0

next[1] = 0

第2次循环:

j=1; k=0

t.data[1]==t.data[0]

满足if条件

j+1=2; k+1=1

next[2] = 1

第3次循环:

j=2; k=1

t.data[2]==t.data[1]

满足if条件

j+1=3; k+1=2

next[3] = 2

第4次循环:

j=3; k=2

t.data[3]!=t.data[2]

不满足if条件

k=next[2]=1

第5次循环:

j=3; k=1

t.data[3]!=t.data[1]

不满足if条件

k=next[1]=0

第6次循环:

j=3; k=0

t.data[3]!=t.data[0]

不满足if条件

k=next[0]=-1

第7次循环:

j=3; k=-1

k=-1满足if条件

j+1=4; k+1=0; next[4] = 0

4.3 串的模式匹配

4.3.2 KMP算法

KMP过程

```
i=0; j=0;
while (s和t都没有扫描完)
{ if (j=-1或者它们所指字符相同)
    i和j分别增1;
  else
    i不变, j回退到j=next[j] (即模式串右滑);
}
if (j超界)
    返回i;                //模式匹配成功
else
    返回-1;                //模式匹配失败
```

4.3 串的模式匹配

4.3.2 KMP算法

KMP算法:

```
int KMPIndex(SqString s, SqString t)
{ int next[MaxSize], i=0, j=0;
  GetNext(t, next);
  while (i<s.length && j<t.length)
  {
    if (j==-1 || s.data[i]==t.data[j])
    { i++;
      j++; //i、j各增1
    }
    else j=next[j]; //i不变, j后退
  }
  if (j>=t.length)
    return(i-t.length); //返回匹配模式串的首字符
  else
    return -1; //返回不匹配标志
}
```

4.3 串的模式匹配

4.3.2 KMP算法

KMP算法分析

- 设串 s 的长度为 n , 串 t 长度为 m 。
- KMP算法中求next数组的时间复杂度为 $O(m)$, 在后面的匹配中因主串 s 的下标不减即不回溯, 比较次数可记为 n , 所以KMP算法平均时间复杂度为 $O(n+m)$ 。

4.3 串的模式匹配

4.3.2 KMP算法

示例已知字符串S为“*abaabaabacacaabaabcc*”，模式串t为“*abaabc*”，采用KMP算法进行匹配，第一次出现“失配”($s[i] \neq t[j]$)时， $i=j=5$ ，则下次开始匹配时， i 和 j 的值分别是（ ）。

A. $i=1, j=0$ B. $i=5, j=0$ C. $i=5, j=2$ D. $i=6, j=2$

说明：本题为2015年全国考研题

j	0	1	2	3	4	5
$t[j]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>c</i>
$next[j]$	-1	0	0	1	1	2

选C

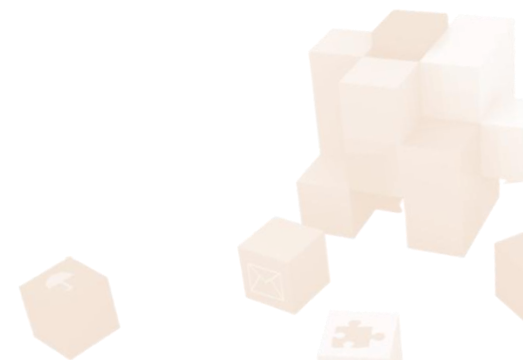
4.3 串的模式匹配

4.3.2 KMP算法

示例 设目标串 $s = \text{"aaabaaaab"}$ ，模式串 $t = \text{"aaaab"}$ 。KMP模式匹配过程。

(1) 求 t 的next

j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3



4.3 串的模式匹配

4.3.2 KMP算法

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
next[j]	-1	0	1	2	3

s: 0 1 2 3 4 5 6 7 8
 a a a b a a a a b

t: 0 1 2 3 4
 a a a a b

失败:

$i=3, j=3$

下一趟:

$j=\text{next}[j]=\text{next}[3]=2$

$i=3$

4.3 串的模式匹配

4.3.2 KMP算法

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
next[j]	-1	0	1	2	3

$i=3$
 $j=2$

$s:$ 0 1 2 3 4 5 6 7 8
 a a a b a a a a b

$t:$ 0 1 2 3 4
 a a a a b



失败:

$i=3, j=2$

下一趟:

$j = \text{next}[j] = \text{next}[2] = 1$

$i=3$

4.3 串的模式匹配

4.3.2 KMP算法

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
next[j]	-1	0	1	2	3

$i=3$
 $j=1$

$s:$ 0 1 2 3 4 5 6 7 8
 a a a b a a a a b

$t:$ 0 1 2 3 4
 a a a b

失败:

$i=3, j=1$

下一趟:

$j = \text{next}[j] = \text{next}[1] = 0$

$i=3$

4.3 串的模式匹配

4.3.2 KMP算法

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
next[j]	-1	0	1	2	3

$i=3$
 $j=0$

$s:$ 0 1 2 3 4 5 6 7 8
 a a a b a a a a b

$t:$ *a a a a b*
 0 1 2 3 4

失败:

$i=3, j=0$

下一趟:

$j=\text{next}[j]=\text{next}[0]=-1$

因为 $j=-1$ 了, 所以

$i++; j++;$

得到 $i=4; j=0$

4.3 串的模式匹配

4.3.2 KMP算法

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
next[j]	-1	0	1	2	3

因为 $j=-1$:

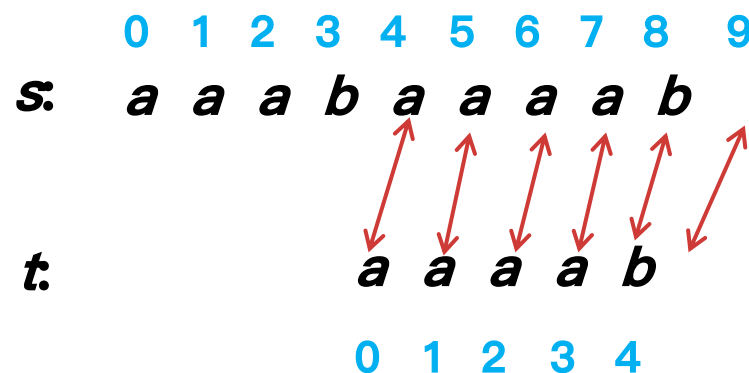
$i++$;

$j++$;

即:

$i=4$

$j=0$



成功匹配:

返回 $i-t.length$

$=9-5=4$

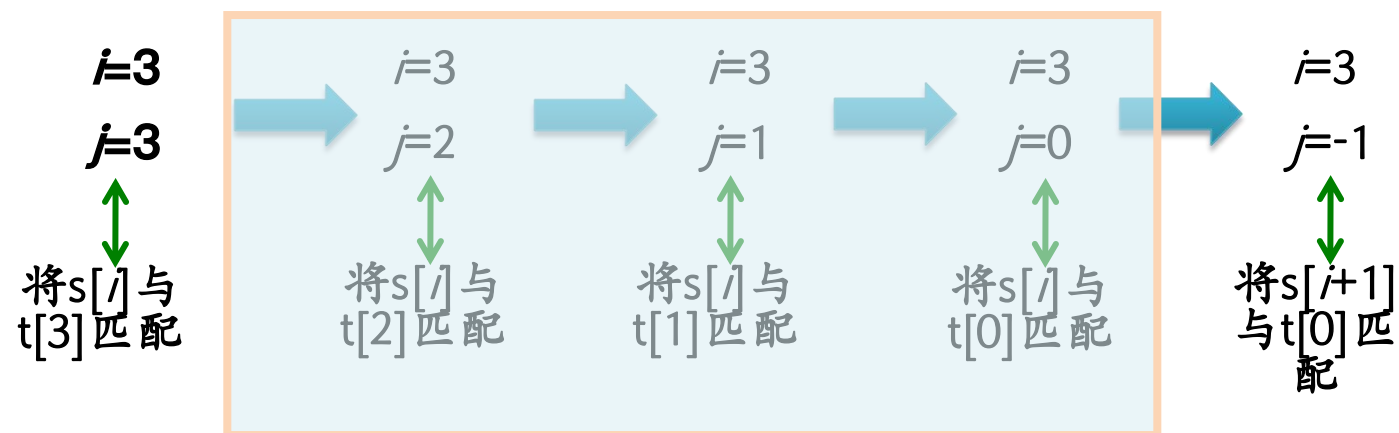
4.3 串的模式匹配

0 1 2 3 4 5 6 7 8
s: *a a a b a a a a b*
t: *a a a a b*

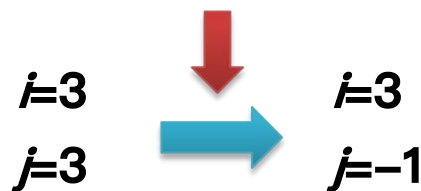
4.3.2 KMP算法

<i>j</i>	0	1	2	3	4
<i>t</i> [<i>j</i>]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
next[<i>j</i>]	-1	0	1	2	3

前面的匹配过程:



因为 *t*[3]=*t*[2]=*t*[1]=*t*[0]='*a*' 是不必要的



4.3 串的模式匹配

4.3.2 KMP算法 KMP算法的改进

将next改为nextval:

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
next[j]	-1	0	1	2	3
nextval[j]	-1	-1	-1	-1	3

next[1]=0

t[1]=t[next[1]]=t[0]='a'

∴ nextval[1]=nextval[0]=-1

t[4]='b' ≠ t[next[4]]=t[3]='a'

∴ nextval[4]=next[4]



- nextval[0]=-1
- 当t[j]=t[next[j]]时: nextval[j]=nextval[next[j]]
- 否则: nextval[j]=next[j]

用nextval取代next, 得到改进的KMP算法。

4.3 串的模式匹配

4.3.2 KMP算法

使用改进后的KMP算法示例：

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
nextval[j]	-1	-1	-1	-1	3

0 1 2 3 4 5 6 7 8
s: a a a b a a a a b
↑ ↑ ↑
t: a a a a b
0 1 2 3 4

失败:

$i=3$

$j=3, j=\text{nextval}[3]=-1$

4.3 串的模式匹配

4.3.2 KMP算法

j	0	1	2	3	4
t[j]	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
nextval[j]	−1	−1	−1	−1	3

因为 $j=-1$:

$j = -1$;
 $i++$;
 $j++$;
 即

	0	1	2	3	4	5	6	7	8
s:	a	a	a	b	a	a	a	a	b
t:					a	a	a	a	b

(Note: In the original image, orange double-headed arrows connect the 'a' characters at indices 4, 5, 6, 7, and 8 of array s to the 'a' characters at indices 0, 1, 2, 3, and 4 of array t, respectively.)

成功匹配:

返回 *i*-t.length

$$=9-5=4$$

改进后的KMP算法进一步提高模式匹配的效率。